DEVELOPER HANDOFF · BUILD PLAYBOOK



Restaurant Billing SaaS

The Kitchen Playbook

Roles, architecture and a day-by-day plan to serve the full SRS in 20 days, before Diwali

SERVICE WINDOW	PLATES	FINAL TASTING
13 Oct – 1 Nov 2026	Standard · Pro · Premium	AC-01 to AC-12

Gagan · Head Chef · Full Stack + DevOps Engineer
Aditya · Sous Chef · Frontend & UI Developer

BASED ON SRS V1.0 · PREPARED 12 OCTOBER 2026

TONIGHT'S SERVICE

The Menu

Twelve courses from first order to final bill

01 Today's Order ..... *project at a glance*

02 The Brigade ..... *Gagan & Aditya's roles*

03 Station Ownership ..... *RACI per module*

04 The Pantry ..... *tech stack*

05 Kitchen Layout ..... *architecture*

06 House Rules ..... *working agreement*

07 Service Schedule ..... *day-by-day 20-day plan*

08 Quality Check at the Pass ..... *acceptance criteria*

09 Today's Menu vs Off-Menu ..... *scope*

10 Chef's Defaults ..... *open decisions*

11 Kitchen Fires ..... *risks & cut line*

12 Closing Shift ..... *handoff & payment*

Based on SRS v1.0 dated 12 Oct 2026 · Prepared 12 Oct 2026

COURSE ONE

01 Today's Order

> We serve the full SRS v1.0 (all three plans plus the Super Admin portal) in 20 working days, 13 Oct to 1 Nov 2026, with 2 to 7 Nov held as buffer before Diwali (8 Nov). Full payment is tied to a live demo of acceptance criteria AC-01 to AC-12.

Item	Detail
Dish	Multi-tenant restaurant billing & management SaaS (SRS v1.0, 12 Oct 2026)
Plates	Standard, Professional, Premium + web Super Admin portal
Tables served	Windows (cashier, hub, KDS) and Android (cashier, waiter, KDS); admin portal in browser
Kitchen	2 people: Gagan (Head Chef: full stack + DevOps), Aditya (Sous Chef: frontend & UI, light backend)
Service window	Day 1 = Tue 13 Oct · Day 20 = Sun 1 Nov · buffer 2 to 7 Nov
Bill	Advance received; balance on sign-off of AC-01 to AC-12

Head Chef's honest read. The SRS is a 10 to 14 week build for a normal team. We can serve it in 20 days only by (1) using one client codebase (Flutter) for every screen, (2) making the restaurant's Windows PC a local hub so offline LAN is the default path, not a bolt-on, (3) getting the client to sign off on our default answers to the open decisions by Day 2, and (4) shipping Premium features at MVP depth. Course 09 spells out what is full-depth and what is not. That list must be agreed in writing before Day 3, because it is what protects the final payment.

COURSE TWO

02 The Brigade

Gagan owns everything behind the pass (data, server, hub, sync, security, infrastructure) and the client relationship. Aditya owns everything the guest and staff see (every screen on Windows, Android and web) plus simple CRUD endpoints under Gagan's review. Whoever owns the API contract owns the behaviour; whoever owns the screen owns the experience.

Gagan, Head Chef

FULL STACK + DEVOPS ENGINEER · TECH LEAD

Mission: a backend and offline engine that never loses an order or double-charges a bill, deployed and monitored in production.

- Architecture, DB schema, permission matrix, API contract (OpenAPI), published before Aditya builds each screen
- Cloud API: tenants, auth, JWT/refresh, RBAC, MFA (TOTP + step-up reset), licensing, entitlements, signed license tokens, audit log
- Branch hub server (Dart, inside the Windows app): orders, tables, bills, payments, idempotency, concurrency, WebSocket bus, device pairing
- Sync engine: outbox, idempotent push, config pull, conflicts, reconciliation report
- Billing math core (paise, GST, discounts, tips, rounding) + golden test vectors
- Printing integration (ESC/POS over LAN + Windows PDF fallback)
- Premium backend: franchise/branches, KDS events, recipe versions, approvals, inventory movements, shifts, franchise invoice
- DevOps: Docker, India-region cloud, managed Postgres, storage, CI/CD, Windows installer + Android builds, backups, Sentry, uptime, secrets
- Critical-path tests: entitlement, RBAC, tenant isolation, money math, offline reconnect, duplicate payment
- Reviews every backend PR (incl. Aditya's); merge authority on `main`
- Client communication: daily note, checkpoint demos, sign-offs, invoice

Decision rights: architecture, data model, security, scope cuts (after client agreement), release go/no-go.

Daily capacity



70% build · 15% review / unblocking Aditya · 15% DevOps + client

Aditya, Sous Chef

FRONTEND & UI DEVELOPER · LIGHT BACKEND

Mission: every role gets a fast, obvious screen. A cashier closes a bill in under 30 seconds; a waiter punches an order one-handed on a phone.

- Design system "House Plating": restaurant palette, typography, spacing, tokens, component kit (buttons, dialogs, keypad, table tiles, order cards, status chips)
- Screen map (handoff deliverable, Day 2) and all wireframes
- Super Admin web portal (Flutter Web): restaurant list/filter, onboarding, license actions, platform admins, MFA enrol/reset, audit viewer
- POS app (Flutter, Windows + Android): login, cashier POS, table map + floors, order taking, bill / payment / tip / reversal dialogs, bill history, receipt layout
- Waiter mode (Android): assigned tables, order entry, modifications, status
- Kitchen Display screens: ticket queue, accept / preparing / ready, timers
- Premium screens: recipe editor, approval inbox, inventory, shifts/breaks, owner and franchise dashboards, reports
- Connection and sync UX: hub status, offline banner, sync indicator, error and empty states
- Light backend (Gagan reviews): menu, category, table, floor CRUD endpoints; seed/demo data
- User documentation with screenshots; UI tests of happy paths
- Reviews UI PRs from Gagan

Decision rights: layout, interaction, visual design and component API; raises an API change request instead of working around the contract.

Daily capacity



85% screens · 10% light backend · 5% docs

Hand-off points between the two stations

1. Gagan publishes the API contract + mock JSON for a module by 12:00 the day before Aditya starts its screens.
2. Aditya builds against mocks and switches to the real endpoint the moment it lands on staging.

3. Any contract change is a PR to `packages/contracts`, tagged for both, merged the same day.
4. Bugs: whoever owns the layer fixes it; if unclear, Gagan triages within 2 hours.

COURSE THREE

03 Station Ownership (RACI)

> Every SRS module has exactly one Accountable owner. R = does the work, A = answers for it, C = consulted, I = informed.

Module (SRS §)	Gagan	Aditya	Done by
Architecture, schema, API contract, permission matrix (§12)	A/R	C	Day 2
Screen map + design system (§12)	C	A/R	Day 2
Tenant onboarding, restaurant lifecycle (§3.1)	A/R backend	R portal UI	Day 3
Auth, RBAC, platform admins, MFA + step-up reset (§3.3)	A/R	R screens	Day 4
Licensing, plans, entitlements (§3.2)	A/R	R screens	Day 4
Audit & security logs (§3.4)	A/R	R viewer	Day 4
Menu, categories, tax rules, images (§4.2)	A, review	R (CRUD + UI)	Day 6
Tables, floors, table transfer (§4.1, §5)	A/R transfer logic	R CRUD + layout UI	Day 9 / 11
Orders, bills, taxes, tips, discounts, reversals (§4.2)	A/R	R screens	Day 9
Printing (§4.2)	A/R driver	R receipt layout	Day 9
Multi-cashier concurrency, duplicate-payment guard (§5)	A/R	C	Day 11
Waiter Android mode (§5)	C	A/R	Day 13
Local LAN hub, pairing, offline queue, cloud sync (§5)	A/R	R status UX	Day 13
Franchise/branches, branch-scoped RBAC, auditor (§6)	A/R	R dashboards	Day 14
Kitchen Display System (§6)	R events	A/R screens	Day 15
Recipes + menu approval workflow (§6)	A/R	R screens	Day 15
Inventory + stock lifecycle (§6)	A/R	R screens	Day 16
Shifts, breaks, attendance (§6)	R	A/R	Day 17
Consolidated franchise invoice (§6)	A/R	R PDF layout	Day 17
Reports & dashboards (§6)	R queries	A/R	Day 17
Security review, load + offline tests (§9, Phase 5)	A/R	C	Day 18
CI/CD, infra, backups, monitoring, installers (§9)	A/R	I	Day 19
User docs + admin ops guide (§12)	A/R ops guide	A/R user docs	Day 19
Client demos, sign-offs, invoice	A/R	C	Each checkpoint

COURSE FOUR

04 The Pantry (Tech Stack)

> One Flutter codebase covers Windows, Android and the web admin portal, so Aditya builds each screen once and the hub server reuses the same Dart business logic. The cloud runs NestJS + PostgreSQL.

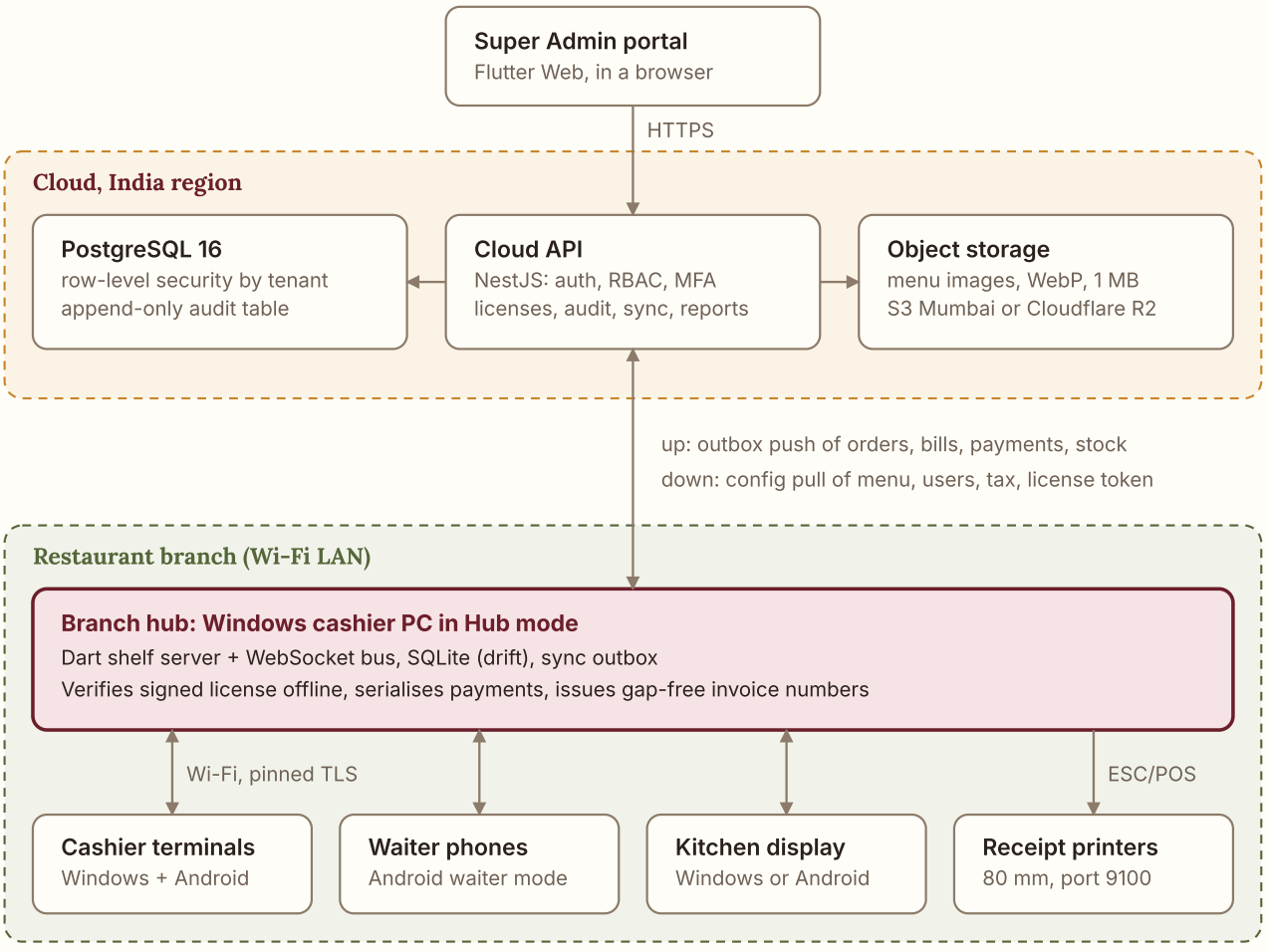
Layer	Choice	Why it fits 20 days	Owner
POS / waiter / KDS app	Flutter 3 (Windows + Android), Riverpod, go_router	One codebase, both target OSes, team already fluent	Aditya
Super Admin portal	Flutter Web (separate app, shared UI kit)	Same components + same stack; SRS allows a browser portal	Aditya
Branch hub server	Dart `shelf` + WebSocket, embedded in the Windows app ("Hub mode")	Shares the billing core with clients; no separate install	Gagan
Local storage	SQLite via `drift` (hub and clients)	Reliable local-first persistence, migrations, typed queries	Gagan
Shared core	Dart package `core`: models, money (integer paise), GST, validation	Same totals on every device; golden test vectors	Gagan
Cloud API	NestJS (TypeScript) + Prisma	Fast CRUD, guards for RBAC/entitlements, OpenAPI generation	Gagan (Aditya: simple CRUD)
Database	PostgreSQL 16, row-level security on `tenant_id`	Tenant isolation enforced in the DB, not only in code	Gagan
Realtime	WebSocket on hub (LAN); Socket.IO on cloud for portal	Live order/table/KDS state across terminals	Gagan
Auth / MFA	Argon2 hashing, JWT (15 min) + rotating refresh, TOTP (`otplib`) + hashed recovery codes	Covers SRS §3.3 and §9 directly	Gagan
License token	Ed25519-signed token (plan, features, limits, expiry), verified offline on hub	Satisfies "do not trust a local license flag"	Gagan
Files / images	S3-compatible storage (AWS S3 Mumbai or Cloudflare R2), 1 MB cap, WebP	Cheap, simple signed uploads	Gagan
Printing	ESC/POS over TCP 9100 (80 mm thermal) + Windows PDF / system print fallback	Works on both OSes without vendor SDKs	Gagan + Aditya
PDFs	`pdf` Dart package (receipts, franchise invoice)	Same layout on all devices	Aditya
Hosting	AWS ap-south-1 (EC2/Lightsail + RDS) or DigitalOcean Bangalore; Docker Compose + Caddy (auto TLS)	India data residency, one-person ops	Gagan
CI/CD	GitHub Actions: lint, tests, API image, Windows installer (MSIX / Inno Setup), Android APK/AAB, Flutter Web deploy	Every merge to `main` lands on staging	Gagan
Observability	Sentry (API + Flutter), Uptime Kuma, structured JSON logs	Errors visible during UAT	Gagan

Layer	Choice	Why it fits 20 days	Owner
Testing	Jest + Supertest (API), Dart `test` (core/hub), Flutter widget + integration tests, k6 (load)	Covers SRS §9 automated test list	Both
Tracking	GitHub Projects board + this playbook	Single source of truth	Gagan

COURSE FIVE

05 Kitchen Layout (Architecture)

> Every branch runs a hub on its Windows cashier PC. All devices in the branch talk to the hub over Wi-Fi, and only the hub talks to the cloud. Offline LAN becomes the normal path, and each branch gets a single writer for live operations, which removes most sync conflicts.



Branch devices talk to the hub; only the hub talks to the cloud. If the hub is switched off, the branch pauses rather than splitting into two versions of the truth.

Who writes what (the rule that keeps sync simple)

Data	Single writer	Flows
Tenants, licenses, users, roles, plans, menu, tax rules, published recipes	Cloud	Cloud → hub (pull by cursor)
Orders, bills, payments, reversals, table transfers, KDS tickets, stock movements, shift records	Branch hub	Hub → cloud (outbox push)

Data	Single writer	Flows
Audit events	Where the action happened	Hub events pushed to cloud; cloud is the permanent store

Key decisions

1. **Tenancy.** Every row carries `tenant_id` (+ `branch_id` where branch-scoped). Postgres RLS uses `SET app.tenant_id` per request; an automated test proves tenant A cannot read tenant B through any endpoint.
2. **Hub mode.** The same Flutter app starts a local `shelf` server when the device is set as hub. Standard plan = one device that is its own hub (Windows or Android), so Standard works offline too.
3. **Pairing.** The hub shows a QR code (IP, port, one-time code, TLS cert fingerprint). Scanning it issues a branch-bound device token. LAN traffic uses a self-signed cert pinned by that fingerprint; mDNS discovery with manual IP fallback.
4. **Concurrency.** The hub serialises writes per order (row version + optimistic check). Every payment carries an idempotency key from the terminal; a repeated key returns the first result, so retries never double-charge (AC-07).
5. **Sync.** Each hub write appends an event (UUIDv7) to `SyncOutbox`. A worker pushes batches to `/sync/push`; the cloud dedupes by event id. Config comes down via `/sync/pull?since=<cursor>`. Rejected events land in `SyncConflict` and a reconciliation report. The UI shows synced / pending / failed counts.
6. **License enforcement.** The cloud issues an Ed25519-signed license token (plan, features, limits, expiry, grace). The hub verifies it offline and refreshes on every sync. Cloud APIs re-check with a `@RequiresFeature()` guard (AC-02).
7. **Money.** Integer paise everywhere; line tax per line, rounded half-up; bill rounding to the nearest rupee as a visible line. Paid bills are never edited; corrections are `BillAdjustment` records.
8. **Invoice numbers.** Issued by the hub per branch per financial year, e.g. `BR01/26-27/000123`, gap-free.
9. **Audit.** `AuditEvent` is append-only: the app DB role has INSERT/SELECT only. No passwords, tokens, MFA secrets or card data are logged.
10. **Hub off.** Other devices show "Kitchen hub offline" and stop taking orders (no split-brain). Mitigation: UPS on the hub PC, auto-start with Windows; a v1.1 option to promote a second device as hub.

COURSE SIX

06 House Rules (Working Agreement)

> One monorepo, contract-first APIs, and nothing reaches `main` without the other chef's review.

Repo layout (melos for Dart, pnpm for TypeScript)

```
restaurant-saas/
  apps/
    pos_app/      Flutter: Windows + Android (cashier, waiter, KDS, owner, hub mode)  Aditya UI / Gagan hub
    admin_web/    Flutter Web: Super Admin portal                                      Aditya
    api/          NestJS cloud API + Prisma schema                                     Gagan
  packages/
    core/         Dart: models, money, GST, validation, golden vectors                Gagan
    hub_server/   Dart: shelf server, drift DB, sync worker                            Gagan
    ui_kit/       Flutter: "House Plating" design system                               Aditya
    contracts/    OpenAPI spec + mock JSON fixtures                                    Gagan
  infra/          Docker, Compose, Caddy, GitHub Actions, backup scripts               Gagan
  docs/           Architecture, permission matrix, screen map, guides                  Both
```

Branching and review

- `main` is protected and auto-deploys to staging. Work happens on `feat/<module>-<short>` branches.
- PRs stay small (under ~400 lines). Backend PRs: Gagan reviews. UI PRs: Aditya reviews Gagan's; Gagan skims Aditya's for API use.
- Review turnaround under 2 hours in working hours. A PR older than half a day gets pinged.
- Release tags `v1.0.0-rcN` from Day 18; production deploys only from tags.

Daily rhythm ("Service")

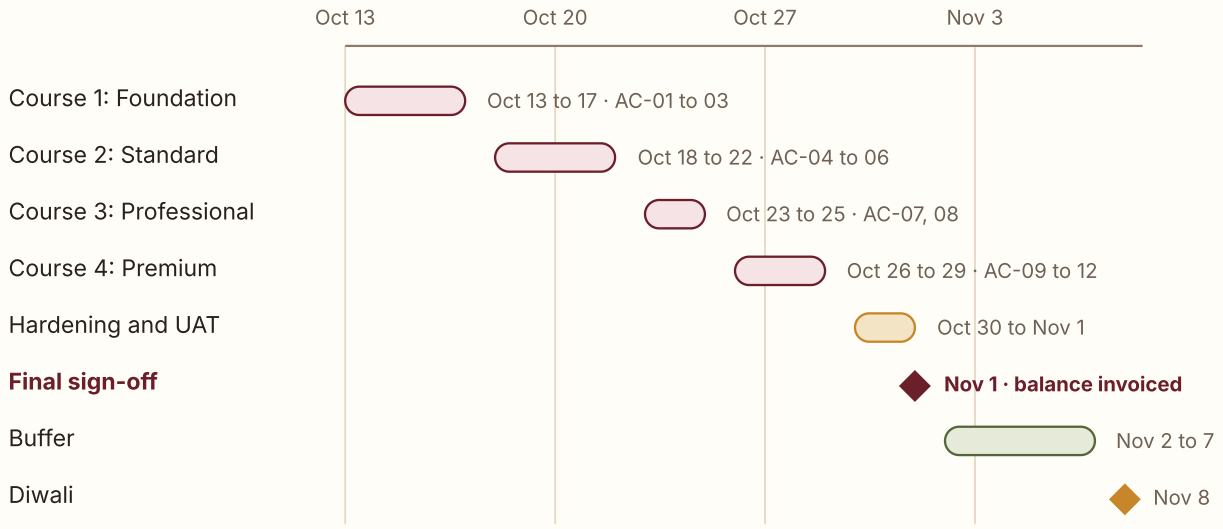
Time (IST)	Ritual	What happens
10:00	Morning Briefing (15 min)	Yesterday done, today's plate, blockers. Check today's Service Schedule row.
12:00	Contract drop	Gagan publishes API contract + mocks for tomorrow's screens.
18:30	Staging tasting	Both click through today's features on staging builds.
20:00	Closing Check	Push everything, update the board; Gagan sends the client a 3-line progress note.

Definition of done (per feature): merged to `main`; deployed to staging; happy-path and one failure-path test pass; audit + permission check present for critical actions (payment, reversal, license, MFA reset, sync); the screen has loading, empty and error states; board card moved to Done.

COURSE SEVEN

07 Service Schedule (20 Days)

> Four courses, each ending in a client tasting: Foundation by Day 5 (17 Oct), Standard by Day 10 (22 Oct), Professional by Day 13 (25 Oct), Premium by Day 17 (29 Oct), then three days of hardening and handoff ending 1 Nov.



Four client tastings by 29 Oct leave a week of buffer before Diwali.

Day	Date	Gagan (Head Chef)	Aditya (Sous Chef)	Tasting / gate	Done
1	Tue 13 Oct	Kickoff call; send Chef's Defaults for sign-off; monorepo, CI skeleton, staging server, Postgres; schema v1	Design system tokens + restaurant palette; screen map; POS wireframes	Defaults sent to client	☐
2	Wed 14 Oct	Architecture doc, permission matrix, OpenAPI v1; auth (Argon2, JWT, refresh), tenants, users, RBAC guards, seed	ui_kit components (buttons, keypad, dialogs, table tile, order card); login screens; portal shell	Handoff pack to client; defaults signed	☐
3	Thu 15 Oct	MFA (TOTP, recovery codes, step-up), platform admins, audit service (append-only)	Portal: restaurant list/search/filter, onboarding form, status actions		☐
4	Fri 16 Oct	Plans, licenses, entitlements guard, signed license token, suspend/block/grace	Portal: license screens, admin users & roles, MFA enrol/reset, audit viewer		☐
5	Sat 17 Oct	Hub server skeleton (shelf + drift), QR pairing, WebSocket bus, license verify on hub	POS app shell, role-based routing, hub pairing screen; menu screens on mocks	**Course 1:** AC-01, 02, 03 demo	☐
6	Sun 18 Oct	Config pull sync (menu, tax, users, tables) cloud → hub; review Aditya's CRUD	Menu/category/table CRUD endpoints (light backend); menu management + image upload UI		☐
7	Mon 19 Oct	`core` billing math + golden vectors; order engine on hub (lines, price snapshots, status history)	Table map (free/occupied), dine-in + takeaway order screen		☐
8	Tue 20 Oct	Bills, payments with idempotency, discounts (permission), tips to waiter, reversals with reason, bill adjustments	Bill screen, payment dialog (cash/UPI/card, pay across modes), tip selector, reversal dialog		☐

Day	Date	Gagan (Head Chef)	Aditya (Sous Chef)	Tasting / gate	Done
9	Wed 21 Oct	Table transfer (atomic, conflict check); ESC/POS + PDF printing	Receipt layout, table transfer UX, bill history + reprint		☐
10	Thu 22 Oct	Outbox push sync, cloud dedupe, reconciliation report, sync status API	Sync status indicator, offline banner, Standard end-to-end polish	**Course 2:** Standard MVP, AC-04 to 06; build sent	☐
11	Fri 23 Oct	Floors, multi-cashier concurrency (row versions), duplicate-payment tests	Floor layout editor, floor switcher, cross-floor transfer		☐
12	Sat 24 Oct	Resilience: hub restart recovery, client retry queue, Wi-Fi drop/reconnect	Waiter mode (Android): assigned tables, order entry, modifications		☐
13	Sun 25 Oct	Offline test harness: WAN cut, LAN cut, duplicate + conflict simulation	Waiter order status, connection states, error/empty states	**Course 3:** Professional, AC-07, 08	☐
14	Mon 26 Oct	Franchise + branches, branch-scoped RBAC, auditor read-only (UI + API)	KDS screens: ticket queue, accept/preparing/ready, timers		☐
15	Tue 27 Oct	KDS events + timestamps; recipe versions; menu approval workflow	Recipe editor, approval inbox, publish flow		☐
16	Wed 28 Oct	Inventory: departments, items, receipts, transfers, adjustments, wastage, recipe deduction at "Ready", low-stock alerts	Inventory screens: stock list, receipt entry, transfer, wastage, low-stock alerts		☐
17	Thu 29 Oct	Shifts/breaks API, consolidated franchise invoice (PDF), report queries	Shift + break screens, owner + franchise dashboards, reports	**Course 4:** Premium, AC-09 to 12	☐
18	Fri 30 Oct	Security pass (RLS tests, rate limits, secret scan, dependency audit), k6 load test, backup + restore drill	UI polish, tablet layouts, accessibility, bug-bash fixes	Release candidate rc1	☐
19	Sat 31 Oct	Production infra, signed Windows installer, Android AAB/APK, monitoring + alerts, ops guide	User guide with screenshots, demo data, client UAT support	**Client UAT** on production	☐
20	Sun 1 Nov	UAT fixes, final full AC-01 to AC-12 demo, handoff pack, invoice	UAT fixes, final screen walkthrough	**Sign-off + final invoice**	☐

Buffer: Mon 2 Nov to Sat 7 Nov for slips and client-requested fixes. Payment is requested on sign-off, not at the end of the buffer.

COURSE EIGHT

08 Quality Check at the Pass

> Each SRS acceptance criterion has one owner, a demo day, and an automated test that runs in CI. The final demo replays all twelve.

AC	Workflow	Owner	Demo day	Automated test (happy + failure path)
AC-01	Tenant onboarding	Gagan	5	Verified email → tenant + license created; unverified or duplicate email rejected; tenant A query returns 0 rows of tenant B
AC-02	License enforcement	Gagan	5	Expired/blocked tenant gets 403 on disallowed API; tampered license token rejected by hub signature check
AC-03	MFA reset	Gagan	5	Unauthorised admin → 403; authorised reset without step-up → rejected; success writes audit + notification
AC-04	Table transfer	Gagan	10	Order moves once with all lines; transfer to an occupied table → conflict; two simultaneous transfers → one wins
AC-05	Item reversal	Gagan	10	No reason → rejected; accepted reversal creates adjustment + audit and updates totals
AC-06	Tip attribution	Aditya (UI) + Gagan	10	Tip saved to selected waiter; appears in tip report; dine-in tip without waiter → prompt
AC-07	Concurrent cashiers	Gagan	13	Two terminals see the same state within 1 s; same idempotency key twice → one payment
AC-08	Offline LAN	Gagan	13	WAN cut: waiter order reaches cashier via hub; reconnect pushes it once (cloud dedupes replayed event)
AC-09	KDS	Aditya (UI) + Gagan	17	Order appears on KDS; accept → preparing → ready visible to waiter and cashier with timestamps
AC-10	Recipe inventory	Gagan	17	Ready ticket deducts mapped ingredients with a `StockMovement`; reversal after Ready → wastage entry
AC-11	Auditor	Gagan	17	Auditor sees authorised records; every write endpoint returns 403 for the auditor role (tested across all routes)
AC-12	Franchise invoice	Gagan (data) + Aditya (PDF)	17	Monthly invoice shows per-branch lines and a consolidated total matching plan prices

COURSE NINE

09 Today's Menu vs Off-Menu (Scope)

> Platform, Standard and Professional ship at full depth; Premium ships every listed feature at MVP depth; a short list moves to v1.1 after Diwali. Get this table approved in writing with the defaults on Day 2.

Plate	Area	What ships by 1 Nov
Full	Platform admin	Onboarding, lifecycle, licenses, platform admins + roles, MFA + step-up reset, audit log, entitlements
Full	Standard	Menu, tables, dine-in/takeaway, taxes, discounts, tips, reversals, adjustments, printing, bill history, offline single device
Full	Professional	Floors, multi-cashier, waiter Android, LAN hub, pairing, offline queue + sync, reconciliation
MVP depth	KDS	Queue, statuses, timestamps, instructions; no station routing (grill/bar)
MVP depth	Recipes + approvals	Recipe versions with ingredients and steps; one approval level (chef → owner publish)

Plate	Area	What ships by 1 Nov
MVP depth	**Inventory**	Quantity tracking per department, receipts, transfers, adjustments, wastage, low-stock alerts, deduction at Ready; fixed base unit per item (g / ml / pcs) with a simple purchase-unit factor
MVP depth	**Shifts**	Shift templates, manual clock-in/out, break records, soft warning on overrun
MVP depth	**Franchise billing**	Monthly invoice PDF with branch breakdown; payment recorded manually by Super Admin
MVP depth	**Reports**	Daily sales, dish quantities, tips by waiter, ingredient consumption, stock movement; on-screen + CSV export
v1.1	**Payments**	Razorpay auto-collection of subscriptions, proration, refund automation
v1.1	**Billing extras**	Split bill by items, partial payments across days
v1.1	**Inventory extras**	FIFO / weighted-average valuation, batch + expiry, supplier management
v1.1	**Other**	Hindi / regional languages, payroll export, hub failover to a second device, iOS

COURSE TEN

10 Chef's Defaults (Open Decisions)

> The SRS says open decisions must be signed off before their work starts. With 20 days we cannot wait, so we build these defaults unless the client changes them by end of Day 2 (Wed 14 Oct). Everything is configurable unless marked otherwise. These are product defaults, not legal or tax advice.

SRS open decision	Our default	Client decision
Plan prices, durations, limits	Client supplies prices; plans store per-plan limits (devices, users, branches) as config	
Payment collection	Super Admin records payments manually (bank / UPI) in v1; gateway in v1.1	
Grace period on expiry	7 days full access, then read-only (reports and history visible, no new orders)	
Tax jurisdiction	India GST: CGST + SGST per item rate (0 / 5 / 12 / 18%), GSTIN on bill	
Payment modes	Cash, UPI, Card recorded (no terminal integration); one bill may be paid across several modes	
Split bill / partial payment	Pay across modes: yes. Split by items: v1.1	
Invoice numbering	Per branch, per financial year, gap-free: `BR01/26-27/000123`	
Discounts	Percentage or flat; cashier needs `discount.apply` permission; reason required above a configurable limit	
Reversal approval	Cashier can reverse unsent items; items already sent to kitchen need owner/manager PIN	
Printers	80 mm ESC/POS network printer; Windows system printer as fallback	
Offline for Standard	Yes: the Standard device runs as its own hub	
LAN server device	The main Windows cashier PC; auto-start with Windows; UPS recommended	
Hub switched off	Other devices pause with "hub offline"; no split-brain	
Chef approval scope	Chef approves recipes and ingredient requirements; price changes approved by Owner / IT Admin; Owner has final publish	
Stock deduction moment	When the kitchen marks the ticket Ready	
Valuation, batches, expiry	Quantities only in v1; valuation + batch/expiry in v1.1	
Attendance source	Manual clock-in/out in app; no biometric; no payroll	
MFA method + recovery	TOTP authenticator app + 10 single-use recovery codes; reset only by authorised Super Admin with step-up	
Audit retention	3 years online, then archive (to confirm against client's compliance needs)	
Data residency + backups	India region; daily snapshots kept 30 days + point-in-time recovery 7 days; target RPO 15 min, RTO 4 h	
Languages	English UI in v1	
Devices	Windows 10/11 64-bit; Android 9+ phones and 8 to 10 inch tablets	
Image limits	1 MB per image after upload, converted to WebP	

COURSE ELEVEN

11 Kitchen Fires (Risks)

> The biggest fire is Gagan's load: he owns the backend, the hub, sync and DevOps. The plan protects him by giving Aditya all screens plus simple CRUD, and by fixing a cut line in advance.

Risk	Likelihood	Impact	Mitigation	Owner
Client delays sign-off on defaults or scope	High	Rework, payment dispute	Send Day 1, deadline Day 2, "silence = approval" stated in writing	Gagan
Offline sync bugs (duplicates, lost orders)	Medium	Kills trust, fails AC-08	Single writer per branch, idempotent events, test harness from Day 13, sync on from Day 10	Gagan
Gagan becomes the review bottleneck	High	Aditya blocked	Contract + mocks a day ahead; 2-hour review SLA; Aditya never waits on a real endpoint	Both
Flutter Windows printer quirks	Medium	Receipts fail at demo	Test with the client's actual printer model by Day 9; PDF fallback	Gagan
Scope creep during UAT	High	Misses Diwali	Every new ask goes to the v1.1 list unless it is a bug against this playbook	Gagan
Windows SmartScreen blocks unsigned installer	Medium	Bad first impression	Buy a code-signing certificate on Day 1 (validation takes days) or ship MSIX with sideload instructions	Gagan
Burnout over 20 straight days	High	Quality drops in week 3	One half-day off each per week, staggered (Gagan Sun 18 Oct PM, Aditya Sat 24 Oct PM, and so on)	Both
One person sick	Low	1 to 3 days lost	Buffer 2 to 7 Nov; docs and contracts in the repo so either can pick up	Both

Cut line (pre-agreed). If Course 3 (Professional) is not green by Sun 25 Oct, Premium moves to this reduced order without renegotiation: KDS and the franchise invoice stay; recipe approvals become single-step; inventory drops transfers between departments; shifts become clock-in/out only. Those items then ship in the 2 to 7 Nov buffer.

COURSE TWELVE

12 Closing Shift (Handoff and Payment)

> The balance is invoiced on 1 Nov against the signed AC-01 to AC-12 demo. Each earlier tasting gets a written "approved" from the client, so the final sign-off is a formality, not a debate.

Before full build (SRS §12), Day 2

- [ ] Architecture diagram (Gagan)
- [ ] API contract / OpenAPI v1 (Gagan)
- [ ] Database schema (Gagan)
- [ ] Permission matrix: role × action × plan (Gagan)
- [ ] Screen map (Aditya)
- [ ] Chef's Defaults + scope table signed by client (Gagan)

Handoff pack, Day 20

- [ ] Source code repo access shared (Gagan)
- [ ] Deployment instructions + infrastructure-as-code (Gagan)
- [ ] Backup / restore procedure with a logged restore drill (Gagan)
- [ ] Monitoring and alerts, with client access (Gagan)
- [ ] Admin operations guide (Gagan)
- [ ] User guides with screenshots for every role (Aditya)
- [ ] Windows installer + Android APK/AAB + portal URL (Gagan)
- [ ] Test report: CI results for all AC tests and offline simulations (Gagan)
- [ ] v1.1 backlog agreed (Gagan)

Payment checkpoints

Checkpoint	Date	Client confirms
Defaults + scope signed	Wed 14 Oct	Written approval of Courses 09 and 10
Course 1: Foundation	Sat 17 Oct	AC-01 to AC-03 approved
Course 2: Standard	Thu 22 Oct	AC-04 to AC-06 approved
Course 3: Professional	Sun 25 Oct	AC-07, AC-08 approved
Course 4: Premium	Thu 29 Oct	AC-09 to AC-12 approved
Final sign-off	Sun 1 Nov	Full demo accepted → balance invoice raised

Gagan, Head Chef (Full Stack + DevOps) · Date

Aditya, Sous Chef (Frontend & UI) · Date